MCTS-MAJ Progress Report

Benchmark Results

Dataset: EvalsBench (30 samples, random_state=42)

EvalsBench Baseline (Vanilla gpt-4o-mini): 84.49%

System Architecture

MCTS-MAJ: Two-Layer Tree Search with Episodic Memory

The system combines Monte Carlo Tree Search (MCTS) with a Memory-Assisted Judge (MAJ) for LLM evaluation. There are three core components:

1. Memory-Assisted Judge (MAJ)

- Neo4j knowledge graph stores past evaluations as nodes: Policy (task), Attempt (response + verdict), Issue (problems found), Fix (solutions), Semantic (abstract categories)
- Three-stage retrieval: contrastive attempts (similar pass/fail examples), similar issues, semantic pattern aggregation
- Memory builds organically — each evaluation is stored and informs future ones

2. MCTS-Judge (Reasoning Layer)

- Decomposes evaluation into multiple subtasks and explores different reasoning paths via tree search
- Each rollout: Select subtask (UCT + LLM self-assessment) → Expand → Evaluate from that perspective → Backpropagate reward
- Reward signal: trajectory majority vote vs independent quick evaluation. Agreement = positive reward.
- Final verdict: best trajectory's majority vote + global evaluation combining all subtask analyses
- Note: results below used hardcoded code-oriented subtasks. Dynamic subtask generation (see Next Steps) has been implemented but not yet benchmarked.

3. MCTS-Retrieval (Retrieval Layer)

- Instead of fixed vector similarity search, uses tree search to explore 7 different retrieval strategies over the memory graph
- Actions: contrastive attempts, similar issues, semantic patterns, and 4 multi-hop traversals (Issues→Fixes, Semantics→Issues, Policy→Attempts, Attempts→Semantics)
- Scores trajectories by relevance, diversity, and volume of retrieved context
- Returns best context to feed into the judge

6 Evaluation Modes (Ablation)

1. **Stateless** — single-pass LLM judge, no memory, no MCTS
2. **MAJ** — single-pass judge with memory retrieval
3. **MCTS-Judge** — tree search reasoning, no memory
4. **MCTS-Judge + Memory** — tree search reasoning with MAJ memory context
5. **MCTS-Retrieval + Judge** — tree search retrieval with single-pass judge
6. **Full MCTS** — MCTS-Retrieval feeds context into MCTS-Judge (novel contribution)

GPT-4o Results (All 6 Modes)

Mode	Accuracy	Avg Latency	Total Time
1. Stateless (no memory, no MCTS)	60.0% (18/30)	3.8s	113s
2. MAJ (memory only)	**63.3% (19/30)**	6.4s	192s
3. MCTS-Judge (no memory)	56.7% (17/30)	126.7s	3802s
4. MCTS-Judge + Memory	60.0% (18/30)	51.2s	1536s
5. MCTS-Retrieval + Standard Judge	60.0% (18/30)	3.4s	102s
6. Full MCTS (Retrieval + Judge)	46.7% (14/30)	48.6s	1459s

GPT-4o-mini Results (3 Modes)

Mode	Accuracy	Avg Latency	Total Time
1. Stateless	63.3% (19/30)	4.5s	135s
2. MAJ (memory only)	60.0% (18/30)	4.9s	147s
3. MCTS-Judge (no memory)	50.0% (15/30)	~120s	945s

Key Findings

Memory helps stronger models

- GPT-4o: Stateless 60.0% → MAJ 63.3% (+3.3%)
- GPT-4o-mini: Stateless 63.3% → MAJ 60.0% (-3.3%)
- Confirms earlier finding: weaker models can't leverage memory effectively

Memory helps MCTS-Judge too

- MCTS-Judge alone: 56.7%
- MCTS-Judge + Memory: 60.0% (+3.3%)
- Memory context improves the tree search reasoning

MCTS-Judge underperforms on QA tasks (with hardcoded subtasks)

- MCTS-Judge (56.7%) is worse than stateless (60.0%)
- These runs used hardcoded subtasks
- Identified as root cause — dynamic subtask generation will be implemented to address this

Full MCTS is worst performer

- Full MCTS (46.7%) is the lowest accuracy
- Combining two layers of MCTS adds noise when memory graph is empty/sparse
- MCTS-Retrieval on empty graph returns no useful context, then MCTS-Judge reasons over nothing

MCTS-Retrieval needs populated memory

- Mode 5 (MCTS-Retrieval + Judge) = 60.0%, same as stateless

- Memory is cleared at start, so MCTS-Retrieval has nothing to explore
- This mode needs pre-seeded or organically built memory to show value

Latency-accuracy tradeoff

- MAJ: best accuracy (63.3%) at only 6.4s — best tradeoff
- MCTS-Judge: 56.7% at 126.7s — worse accuracy, 33x slower
- Full MCTS: 46.7% at 48.6s — worst accuracy, 13x slower

MCTS Parameters Used

Mode	Rollouts	Depth
MCTS-Judge (mode 3)	4	5
MCTS-Judge + Memory (mode 4)	2	3
MCTS-Retrieval (mode 5)	2	2
Full MCTS (mode 6)	2 retrieval + 2 judge	2 + 3

Next Steps

- Dynamic subtask generation — LLM will generates task-specific evaluation perspectives instead of hardcoded subtasks. Rerun benchmarks to measure impact.
- Test on complex evaluation dataset where multi-perspective MCTS decomposition is a natural fit
- Run with pre-seeded memory (100 ground truth examples) to test MCTS-Retrieval with populated graph